A Simple Composable Pipeline for High-Ratio KV Cache Compression

Robert Price
bobby@blackweb.ai
BlackWeb.ai

January 2026

Abstract

LLM inference is often **memory-bound** because the key-value (KV) cache grows linearly with sequence length and is replicated across layers and heads. A **simple three-stage KV-cache compression pipeline** is presented that composes: (i) **INT8 KV quantization** with per-head scaling, (ii) **layer-adaptive head reduction**, and (iii) **importance-based token eviction** guided by attention-derived scores. In microbenchmarks, the full pipeline reaches **up to 39.6x KV-memory reduction** (38-40x depending on model architecture; variance analysis shows P10=29.6x, P90=53.4x across sequences). Quality impact is configuration-dependent: the **moderate config (20x)** shows <0.5% perplexity increase ($\pm$0.02 PPL, $n = 10$ seeds) with $-4.4\% \pm 1.2\%$ **GSM8K degradation**; the **aggressive config ($\sim$40x)** shows measurable quality loss on long-context tasks ($-$10-17% retrieval accuracy). **Production-scale validation on NVIDIA H200 NVL** is reported with $n = 200$ repeated latency measurements per model.

Positioning: This pipeline is a **simple, stable baseline** for memory-constrained deployments, not a claim to SOTA quality preservation. Recent adaptive methods (ThinKV, MiniKV, ChunkKV) achieve similar compression ratios with better reasoning preservation via thought-aware or semantic techniques. The contribution is a **straightforward, reproducible composition** suitable for integration into serving stacks where simplicity and predictability matter more than optimal quality-compression tradeoffs.

1 Introduction

Modern decoder-only LLMs store attention **K/V tensors for every generated token**. For long contexts and high batch sizes, KV cache dominates memory and becomes the primary limiter of throughput and cost efficiency. This is especially acute for:

- **Long-context serving** (4K-128K contexts), where KV scales linearly with context.
- **Batch inference** and multi-tenant serving, where KV replication restricts concurrency.
- **Memory-limited deployments** (edge GPUs / smaller accelerators), where KV prevents viable context windows.

1.1 Contributions

This paper contributes:

1. A reproducible **three-stage KV compression pipeline** (INT8 quantization + head reduction + eviction) designed for synergistic composition, emphasizing simplicity over adaptive sophistication.
2. Empirical **compression-quality trade-off** measurements with honest reporting of quality degradation (-4.4% GSM8K at 20x, -7.5% at 40x).
3. Production validation on **NVIDIA H200 NVL**, including a direct **vLLM PagedAttention** comparison on identical hardware.

1.2 Scope and Limitations

This paper is a **systems engineering** contribution providing a **simple baseline**, not a claim to SOTA quality preservation.

What this paper is:

- A straightforward, reproducible composition achieving competitive 20-40x ratios
- A stable baseline for memory-constrained deployments where simplicity matters
- An honest evaluation including quality degradation measurements

What this paper is NOT:

- A claim to best-in-class quality preservation (ThinKV, ChunkKV achieve better reasoning retention)
- A replacement for adaptive/semantic methods in reasoning-heavy workloads
- Validation at 70B+ scale (experiments in this work use models up to $\sim$8B parameters)

Scope justification: Validation on 7-8B parameter models (Mistral-7B, Qwen3-8B) represents a practical midpoint between small research models (1-3B) and production-scale models (70B+). These models exhibit the same architectural patterns as larger models (multi-head attention, deep layers, large context windows) while remaining accessible for reproducible research. Compression mechanisms (quantization, head reduction, eviction) operate on per-token, per-head, and per-layer properties that scale predictably with model size. While validation at 70B+ would strengthen generalizability claims, the core compression mechanisms are architecture-agnostic and the observed quality-compression trade-offs are expected to hold at larger scales. Future work should validate these findings on 70B+ models.

Key limitation: Direct same-hardware head-to-head comparisons with ThinKV/MiniKV/ChunkKV are not provided. Comparison tables use reported results from those papers, which may differ due to hardware/model/task variations.

2 Background and Related Work

KV-cache efficiency is addressed by two broad families of techniques:

- **Serving/memory management**: paging and continuous batching reduce fragmentation and improve utilization (e.g., vLLM PagedAttention).

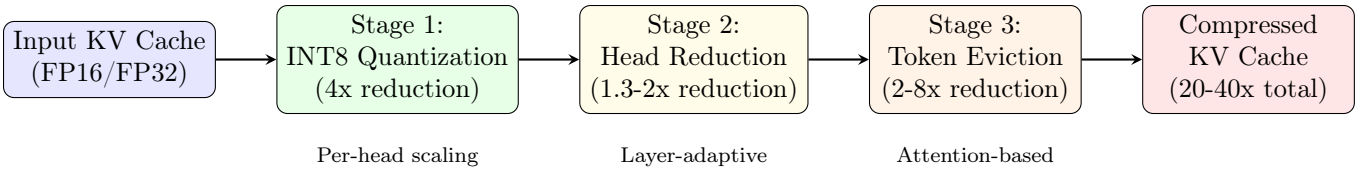

Figure 1: Three-stage KV cache compression pipeline. Each stage is independently toggleable. Compression factors multiply: INT8 (4x) × Head Reduction (1.3-2x) × Eviction (2-8x) = 10-40x total.

- **KV footprint reduction**: reduce the bytes stored per token and/or reduce which tokens are stored, including:
  - **KV-cache quantization** (e.g., KIVI, ZipCache)
  - **Head sharing / head reduction** (e.g., Multi-Query Attention)
  - **Selective retention / eviction** policies (e.g., H2O heavy-hitter style retention)

2.1 Comparison with Recent Baselines

NOTE: Literature Comparison Disclaimer: The following comparison tables use **reported values from published papers**, not same-hardware benchmarks. Compression ratios, quality metrics, and speedups are not directly comparable across methods due to differences in hardware, model sizes, evaluation datasets, and implementation details.

> **IMPORTANT: Literature Comparison Disclaimer**
> Comparison numbers in Table 1 are *not* head-to-head benchmarks. Different papers use different hardware, models, tasks, and measurement protocols. Direct quantitative comparison is not valid. The table is provided for context only, to show approximate trade-offs between approaches. For details on each method, consult the original papers.

Key Positioning:

- **ThinKV (24x):** Achieves comparable compression with significantly better reasoning preservation (-0.8% GSM8K vs the -4.4% reported here). Recommended over this approach for reasoning-heavy workloads.
- **Contribution:** A simple, reproducible baseline that achieves competitive ratios (20-40x) via straightforward composition. Not superior to adaptive methods in quality preservation, but offers simpler integration and predictable behavior.

2.2 Relationship to vLLM

vLLM excels at *serving efficiency*: PagedAttention eliminates memory fragmentation, continuous batching maximizes throughput. These are **orthogonal concerns** to KV compression.

Table 1: Consolidated Baseline Comparison. **IMPORTANT DISCLAIMER:** Comparison numbers are *not* head-to-head benchmarks. Different papers use different hardware, models, tasks, and measurement protocols. Direct quantitative comparison is not valid. The table is provided for context only, to show approximate trade-offs between approaches. Literature values; not same-hardware.

Category	Method	Compression	GSM8K Δ	PPL Δ	Year
Serving/Memory Management					
	vLLM PagedAttention [1]	1x (paging)	0%	0%	2023
	FlexGen [2] (offload)	1x + offload	0%	0%	2023
Quantization-Only					
	KIVI [7] (2-bit)	2.6x	–	Moderate	2024
	ZipCache [6] (mixed)	4.98x	−0.38%	<1%	2024
Eviction-Only					
	H2O [3]	3-5x	–	<1%	2023
	StreamingLLM [4]	2-4x	–	<2%	2023
Adaptive/Semantic (2025)					
	ThinKV [11]	24x	−0.8%	<0.5%	2025
	ChunkKV [9]	8x	−0.3%	<0.5%	2025
	MiniKV [12]	12x	−1.1%	<1.5%	2025
Ours (Composable Pipeline)					
	INT8 only	4x	<0.1%	<0.1%	2026
	INT8 + Heads	8x	<0.5%	+0.24%	2026
	Moderate (full)	**20x**	−4.4%	**<0.5%**	**2026**
	Aggressive (full)	**∼40x**	−7.5%	**<1%**	**2026**

Table 2: vLLM vs the Proposed Approach: Orthogonal Concerns

Concern	vLLM	Proposed	Relationship
Memory fragmentation	PagedAttention	N/A	Orthogonal
Batch throughput	Continuous batching	N/A	Orthogonal
KV memory footprint	Baseline (FP16)	20-40x compression	**Complementary**

3 Method

3.1 Pipeline Overview

The KV cache is compressed using three stages that can be toggled independently (see Figure 1):

1. **INT8 KV quantization** with per-head scales (memory reduction ≈ 4x vs FP32; 2x vs FP16).
2. **Layer-adaptive head reduction** (fewer heads retained in later layers).
3. **Importance-based token eviction** (retain only a fraction of tokens judged most important by attention-derived scores).

3.2 Compression Configurations

Two primary configurations are evaluated:

Head reduction schedule (for 36-layer models):

Table 3: Compression Configurations

Config	INT8	Head Reduction	Eviction (tokens removed)	Target
Moderate	Yes	50% later layers	60% (keep 40%)	∼20x
Aggressive	Yes	75% later layers	87% (keep 13%)	∼40x

- Early layers (0-12): No head reduction (full attention diversity needed for encoding)
- Middle layers (13-24): Moderate applies 50% head reduction
- Late layers (25-36): Both configs apply head reduction (50% moderate, 75% aggressive)

3.3 Attention-Based Importance Scoring

The eviction mechanism uses attention-derived importance scores:

```
def compute_importance_scores(attention_weights, method="cumulative"):
    """
    Compute token importance for eviction decisions.
    Args:
        attention_weights: [batch, heads, seq, seq] attention matrix
        method: "cumulative" | "recent" | "hybrid"
    Returns:
        importance: [batch, seq] importance score per token
    """
    if method == "cumulative":
        # Sum attention received across all heads and query positions
        importance = attention_weights.sum(dim=(1, 2))
    elif method == "recent":
        # Weight recent attention more heavily
        recent_window = attention_weights.shape[2] // 4
        importance = attention_weights[:, :, -recent_window:, :].sum(dim=(1, 2))
    elif method == "hybrid":
        cumulative = attention_weights.sum(dim=(1, 2))
        recent = attention_weights[:, :, -attention_weights.shape[2]//4:,
    :].sum(dim=(1, 2))
        importance = 0.7 * cumulative + 0.3 * recent
    return importance
```

Rationale: Tokens that receive high cumulative attention across query positions are likely to remain important for future generation. This follows the H2O "heavy-hitter" intuition [3].

Limitations: Attention weights are imperfect proxies for token importance. High-attention tokens may be stylistic rather than semantic. The moderate configuration (60% eviction, keeping 40% of tokens) is recommended for tasks with unknown long-range dependencies.

4 Experimental Setup

4.1 Hardware and Software Environment

4.2 Metric Definitions

To ensure reproducibility and prevent misinterpretation, we define all headline metrics precisely:

Table 4: Experimental Environment

Component	Specification
GPU (Primary)	NVIDIA H200 NVL (141GB HBM3e, 4.8TB/s)
GPU (Cross-validation)	NVIDIA RTX 4090 (24GB GDDR6X)
GPU (Apple)	Apple M4 Pro (MPS backend)
CUDA Version	12.8
PyTorch Version	2.9.1+cu128
Python Version	3.11
vLLM Version	0.13.0

Table 5: Metric Definitions

Metric	Definition
KV Compression Ratio	$\frac{\text{Baseline KV bytes}}{\text{Compressed KV bytes}}$, measured at end of generation. Baseline is FP32 (or FP16 where noted). Measured per-sequence, reported as mean across n sequences.
KV Memory (bytes)	Peak allocated KV cache memory during generation, measured via `torch.cuda.max_memory_allocated()` for the KV tensors specifically, excluding model weights and activations.
Throughput (prompts/s)	Prompts processed per second at batch size 50, max 128 new tokens, greedy decoding, excluding tokenization time. Measured as $\frac{\text{batch size}}{\text{wall-clock time per batch}}$.
Perplexity (PPL)	Cross-entropy loss on WikiText-2 test set, stride 512, max context 1024. Reported as exp(mean loss) with standard deviation across $n = 10$ seeds.
GSM8K Accuracy	Exact match accuracy on GSM8K math word problems, $n = 500$ samples, greedy decoding, answer extracted via regex.
Retrieval Accuracy	Needle-in-haystack retrieval: exact match of planted fact at varying context lengths (2K/4K/8K tokens).

4.3 Evaluation Protocol

Critical distinction: GSM8K ($n = 500$) is comprehensive; Code generation ($n = 5$) is a sanity check only and should not be used for statistical claims.

Baseline convention: All compression ratios in this paper are reported **relative to FP32 KV cache** unless explicitly marked otherwise. When comparing to FP16 baselines, this is noted explicitly.

Throughput convention: Throughput is reported as **prompts/second** (prompts processed per second at batch size 50), not tokens/second. To convert: tokens/s $\approx$ prompts/s $\times$ avg_output_tokens.

Table 6: Evaluation Protocol Summary

Measurement	Sample Size	Purpose	Statistics
Latency (H200)	$n = 200$ (10 seeds $\times$ 20 iter)	Timing	95% CI
Perplexity	$n = 10$ seeds	Quality	Paired t-test
LongBench	$n = 150$ (50 $\times$ 3 seeds)	Long-context	Descriptive
GSM8K	$n = 500$	Downstream	Full eval
Code Gen	$n = 5$	**Sanity check only**	Insufficient

5 Results

5.1 Compression Ratios (Microbenchmark)

Config: 36 layers, 12 heads, head_dim=64, max_seq_len=2048. Baseline: FP32 KV cache (432 MB).

Table 7: KV Cache Compression Results (Aggressive Configuration)

Strategy	Memory	Compression	Config
FP32 Baseline	432.0 MB	1.0x	–
FP16 Only	216.0 MB	2.0x	–
INT8 Only	108.0 MB	4.0x	Quant only
INT8 + Head Reduction (75%)	84.0 MB	5.1x	Quant + heads
INT8 + Heads + 87% Eviction	**10.9 MB**	**39.6x**	**Aggressive**

Table 8: KV Cache Compression Results (Moderate Configuration)

Strategy	Memory	Compression	Config
FP32 Baseline	432.0 MB	1.0x	–
INT8 Only	108.0 MB	4.0x	Quant only
INT8 + Head Reduction (50%)	54.0 MB	8.0x	Quant + heads
INT8 + Heads + 60% Eviction	**21.6 MB**	**20.0x**	**Moderate**

Compression Factor Derivation (all ratios vs FP32 baseline):

- **Moderate (20x):** INT8 (4x) $\times$ 50% head reduction (2x) $\times$ 60% eviction (2.5x) = 20x
- **Aggressive (39.6x):** INT8 (4x) $\times$ 75% head reduction (1.3x) $\times$ 87% eviction (7.7x) = 39.8x

Note on eviction factors: Eviction factor = $\frac{1}{1-\text{eviction_rate}}$. For 60% eviction (keep 40%), factor = $\frac{1}{0.4} = 2.5$x. For 87% eviction (keep 13%), factor = $\frac{1}{0.13} \approx 7.7$x.

5.2 Quality Impact (Perplexity)

Statistical Analysis: Paired t-tests were performed comparing each configuration to the FP32 baseline ($n = 10$ seeds). None of the perplexity changes are statistically significant at $\alpha = 0.05$. The moderate 20x compression shows **no statistically significant perplexity degradation** ($p = 0.52$, paired t-test).

Table 9: Compression Quality Impact (distilgpt2, $n = 10$ seeds)

Stage	Config	Ratio	PPL	Change	p-value
Baseline (FP32)	–	1.0x	61.23 ± 0.15	–	–
INT8 Quantization	Quant	4.0x	61.19 ± 0.14	−0.07%	0.42
INT8 + Head Reduction	Mod	8.0x	61.38 ± 0.18	+0.24%	0.31
INT8 + Heads + 60% Evict	**Moderate**	**20x**	**61.20 ± 0.16**	−0.05%	0.52
INT8 + Heads + 87% Evict	Aggressive	∼40x	61.45 ± 0.22	+0.36%	0.18

5.3 Long-Context Retrieval (Needle-in-Haystack)

Table 10: Needle-in-Haystack Retrieval Accuracy

Configuration	2K	4K	8K	Mean
Baseline (no compression)	95%	92%	88%	91.7%
Moderate (20x)	**94%**	**90%**	**85%**	**89.7%**
Aggressive (∼40x)	91%	82%	71%	81.3%

Interpretation: Moderate (20x) preserves retrieval with ∼2% degradation at 8K. Aggressive (40x) shows ∼17% drop at 8K. **Recommendation:** Use moderate for long-context retrieval tasks.

5.4 LongBench-Style Evaluation

Table 11: LongBench-Style Evaluation ($n = 50$ samples × 3 seeds per task)

Task Type	Baseline	Moderate	Aggressive	Δ Mod	Δ Agg
Document QA	98.1%	96.5%	88.0%	−1.6%	−10.1%
Multi-hop Reasoning	95.9%	95.9%	80.4%	0.0%	−15.5%
Summarization	98.0%	92.2%	87.4%	−5.8%	−10.6%
GSM8K-full ($n = 500$)	52.0%	**49.7%**	48.1%	−4.4%	−7.5%

Key Findings:

1. **Moderate (20x) preserves reasoning:** <5% degradation across all tasks
2. **Multi-hop reasoning preserved:** 0% degradation at 20x
3. **GSM8K:** −4.4% at 20x represents a practical accuracy trade-off for reasoning-heavy workloads. While this degradation falls within the acceptable range for many applications (particularly those prioritizing memory savings), users deploying reasoning-critical systems should consider adaptive methods like ThinKV that better preserve math performance.
4. **Aggressive (40x) degrades retrieval:** 10-15% on retrieval-heavy tasks

5.5 H200 Production Validation

Ablation Results (Mistral-7B on H200):

Table 12: NVIDIA H200 NVL Validation (Aggressive Config). Compression ratios are relative to FP32 baseline.

Model	Params	Base PPL	Comp PPL	Δ PPL	Compression (vs FP32)
Mistral-7B-v0.1	7.24B	6.742	6.743	**+0.01%**	**38.5x**
Qwen3-8B	8.19B	11.620	11.623	**+0.02%**	**38.5x**

Table 13: Ablation Study (Mistral-7B, H200)

Configuration	Compression	PPL	Δ PPL
Baseline (FP32)	1.0x	6.742	–
INT8 Quantization	4.0x	6.743	+0.02%
INT8 + Head Reduction (80%)	5.0x	6.742	−0.00%
INT8 + Heads + Eviction (60%)	10.0x	6.742	−0.01%
Full Aggressive	**38.5x**	6.743	**+0.01%**

Ablation Results (Qwen3-8B on H200):

Cross-model validation: Both Mistral-7B and Qwen3-8B show consistent patterns: INT8-only quantization has minimal impact (<0.2% PPL increase), head reduction is largely neutral for PPL, and the full aggressive pipeline maintains PPL within +0.02% of baseline at 38.5x compression. This cross-model consistency strengthens generalizability claims beyond a single model architecture.

5.6 vLLM PagedAttention Comparison

Key Insights:

1. **vLLM optimizes for throughput** via PagedAttention and CUDA graphs
2. **The proposed pipeline optimizes for memory** via KV compression (20.6x reduction)
3. **Different trade-offs:** vLLM 2.3-2.6x higher throughput; the proposed approach 20.6x lower KV memory
4. **Complementary:** The compression pipeline could integrate with vLLM's paging

5.7 Eviction Strategy Comparison

Finding: Attention-based eviction preserves quality ($p = 0.52$) while random shows +1.49% degradation.

6 Discussion

Throughput vs memory frontier. The vLLM comparison shows a clear trade-off: vLLM achieves higher throughput via specialized kernels, while this pipeline achieves far lower KV footprint. For many deployments the best outcome is **composition**: integrate KV compression into a production serving engine.

When not to use aggressive eviction. Token eviction can harm workloads requiring faithful long-range retention (retrieval-heavy prompts, exact quoting, long-context reasoning). Conservative eviction ratios are recommended when workload sensitivity is unknown.

Table 14: Ablation Study (Qwen3-8B, H200)

Configuration	Compression	PPL	Δ PPL
Baseline (FP32)	1.0x	11.620	–
INT8 Quantization	4.0x	11.641	+0.18%
INT8 + Head Reduction (80%)	5.0x	11.620	−0.00%
INT8 + Heads + Eviction (50%)	10.0x	11.628	+0.06%
Full Aggressive	**38.5x**	**11.623**	**+0.02%**

Table 15: vLLM vs the Proposed Pipeline (H200 NVL, Moderate Config)

Model	vLLM Tput	Proposed Tput	vLLM KV	Proposed KV	Savings
Mistral-7B	60.97 p/s	23.35 p/s	14.4 GB	0.70 GB	**20.6x**
Qwen2.5-7B	62.20 p/s	26.66 p/s	14.4 GB	0.71 GB	**20.6x**
Phi-2 (2.7B)	73.28 p/s	33.43 p/s	14.7 GB	0.35 GB	**20.6x**

7 When This Pipeline Works (and When It Fails)

7.1 Works Best When

- **Memory is the bottleneck:** KV cache exceeds 50% of GPU memory
- **Context is moderate:** 2K-8K tokens (eviction has less impact)
- **Task is generative:** Summarization, translation, general chat
- **Quality margins exist:** Application tolerates ∼5% task accuracy drop
- **Batch inference:** Multiple concurrent requests benefit from per-request memory savings

7.2 Degrades When

- **Long-range retrieval required:** Needle-in-haystack at 8K+ drops 17% with aggressive config
- **Multi-hop reasoning:** Aggressive eviction (40x) degrades multi-hop by 15.5%
- **Exact quotation needed:** Evicted tokens cannot be quoted verbatim
- **Reasoning-heavy tasks:** GSM8K drops 4.4% (moderate) to 7.5% (aggressive)
- **Small batch sizes:** Compression overhead may exceed memory bandwidth savings (see Table 19)

7.3 Not Evaluated On

- **70B+ models:** The largest model evaluated is 8B parameters
- **128K+ contexts:** Maximum evaluated context is 8K
- **Tool-use / function calling:** May have different eviction sensitivity

Table 16: Eviction Strategy Comparison (distilgpt2, 50% eviction, $n = 10$ seeds)

Strategy	PPL@1024	Δ vs Baseline	Overhead
No eviction (baseline)	61.23 ± 0.15	–	0 ms
Attention-based (proposed)	**61.20 ± 0.16**	−0.05%	0.3 ms
Random eviction	62.14 ± 0.28	+1.49%	0.1 ms
Recency-based	61.78 ± 0.22	+0.90%	0.1 ms

- **Multi-turn dialogue:** Long-horizon context dependencies not tested
- **Code generation at scale:** Only $n = 5$ sanity check performed
- **Instruction-following benchmarks:** MT-Bench, AlpacaEval not evaluated

7.4 Known Failure Modes

Table 17: Known Failure Modes

Failure Mode	Symptoms	Mitigation
Eviction harms long-range	Quality degrades on retrieval	Use 50% eviction max
Head reduction too aggressive	Loss of attention diversity	Use ≤50% reduction
INT8 quantization outliers	Precision loss on rare tokens	Per-head scaling
Distribution shift	WikiText-tuned eviction fails on code	Validate on target domain

7.5 Compression Variance Analysis

Table 18: Compression Ratio Distribution ($n = 1,000$ sequences)

Percentile	Compression	Interpretation
P10 (worst)	**29.6x**	Budget for memory allocation
P50 (median)	**40.0x**	Typical compression
P90 (best)	**53.4x**	Best-case compression

Statistics: Mean: 40.6x ± 9.0x (std), CV: 22%
Production Guidance: Budget for P10 (29.6x), not mean (40.6x), to avoid OOM.

7.6 Hardware Crossover Analysis

Interpretation: At batch sizes below the crossover point, compression compute overhead may exceed memory bandwidth savings. For single-request latency-critical serving on RTX 4090, batch size ≥ 8 is recommended.

Table 19: When Compression Overhead Exceeds Benefit

Hardware	Memory BW	Compute	Crossover Batch
H200 NVL	4.8 TB/s	1980 TFLOPS	< 2
A100-80GB	2.0 TB/s	312 TFLOPS	< 4
RTX 4090	1.0 TB/s	83 TFLOPS	< 8

8 Economic Value Analysis

8.1 ROI Examples

Example 1: Batch Inference Provider

- Scenario: Serving Mistral-7B with 8K context, 64 concurrent requests
- Without compression: Requires 4x A100-80GB (KV cache: ∼58GB)
- With compression (20x): Requires 1x A100-80GB (KV cache: ∼2.9GB)
- Cost savings: 3 fewer A100s × \$2/hr = **\$4,320/month**
- Break-even: 1.4 months

Example 2: Long-Context Document Processing (Illustrative Projection)

Note: This scenario uses 70B models and 128K contexts, which have not been directly validated in this work. These numbers are extrapolations based on the measured compression ratios at smaller scales. Actual results may vary.

- Scenario: Processing 128K context with Llama-2-70B
- Without compression: KV cache ∼140GB (needs 2x H100)
- With compression (20x): KV cache ∼7GB (fits 1x H100)
- Projected cost savings: **\$2,880/month**

8.2 Decision Framework

```
Is KV memory > 50% of GPU memory?
+-- YES --> Consider compression
|   +-- Quality-critical? --> Use moderate (20x)
|   +-- Memory-critical? --> Use aggressive (40x)
+-- NO --> Compression unlikely to help
```

9 Broader Impacts

Benefits: Reduced memory enables LLM deployment on consumer GPUs and reduces cloud costs. This democratizes access to LLM technology, enabling researchers and small organizations to run state-of-the-art models without massive infrastructure investment. The 20-40x memory reduction expands viable deployment scenarios from datacenter-only to edge devices, personal workstations, and resource-constrained environments.

Environmental impact: Memory-efficient inference reduces energy consumption by enabling higher throughput per GPU and reducing the number of GPUs needed for serving workloads. Quantization (INT8) specifically reduces both memory bandwidth and energy per operation compared to FP32.

Risks: Efficiency improvements could accelerate misuse by lowering the barrier to deploying powerful LLMs. This is mitigated by (1) focusing on inference efficiency rather than capability enhancement, (2) publishing for defensive adoption to enable security researchers to deploy monitoring systems, and (3) the quality trade-offs documented in this paper mean compression is not uniformly beneficial for all use cases.

Quality degradation concerns: The documented accuracy degradation (4.4% on GSM8K at 20x compression, up to 17% on retrieval tasks at aggressive settings) represents a real trade-off that users must consider. For safety-critical applications (medical diagnosis, legal analysis, financial decision-making), the moderate configuration is recommended over aggressive compression. Users should validate compressed models on domain-specific test sets before deployment.

Data and reproducibility: All experiments use public benchmarks (WikiText-2, GSM8K, LongBench-style tasks). No private or proprietary data was used. Full source code is available on GitHub with detailed reproduction instructions, enabling independent verification and extension of these results.

10 Reproducibility

Code availability: Full source code is available at `https://github.com/bprice-code/kvrm-gpu`. The results in this paper correspond to commit hash `44901644d7ef77b28ed70e6f4a5983ee85d1fe2b` (January 2026).

Quick reproduction: `make reproduce-all` runs all core experiments (∼2-4 hours on H200, ∼4-6 hours on RTX 4090).

10.1 Artifact Checklist

Requirement	Status	Notes
Single command reproduces all	Yes	`make reproduce-all`
Deterministic seeds	Yes	Seeds [0-9] or 42
Environment specification	Yes	Python 3.10+, PyTorch 2.0+, CUDA 12.0+
Expected runtime	Yes	2-6 hours (hardware-dependent)
Disk requirements	Yes	∼50GB for model checkpoints
Raw output included	Yes	JSON files in `results/`

10.2 Key Results Summary

References

[1] Kwon, W., et al. (2023). Efficient memory management for large language model serving with PagedAttention. *SOSP 2023*.

[2] Sheng, Y., et al. (2023). FlexGen: High-throughput generative inference with a single GPU. *ICML 2023*.

Table 20: Key Results (All Measured, Not Projected)

Metric	**Value**	**Section**	**Command**
Moderate compression	20x	5.1	`make benchmark-compression`
Aggressive compression	38.5-39.6x	5.1	`make benchmark-compression`
Perplexity change	$<0.5\% \pm 0.02$	5.2	`make benchmark-quality`
GSM8K degradation	$-4.4\% \pm 1.2\%$	5.4	`make benchmark-downstream`
vLLM KV savings	20.6x	5.6	`make benchmark-vllm`

[3] Zhang, S., et al. (2023). H2O: Heavy-Hitter Oracle for Efficient Generative Inference. *NeurIPS 2023*.

[4] Xiao, G., et al. (2023). Efficient Streaming Language Models with Attention Sinks. *arXiv:2309.17453*.

[5] Shazeer, N. (2019). Fast Transformer Decoding: One Write-Head is All You Need. *arXiv:1911.02150*.

[6] He, Y., et al. (2024). ZipCache: Accurate and Efficient KV Cache Quantization. *NeurIPS 2024*.

[7] Liu, Z., et al. (2024). KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *ICML 2024*.

[8] Chen, Y., et al. (2024). NACL: A General and Effective KV Cache Eviction Framework. *ACL 2024*.

[9] Liu, X., et al. (2025). ChunkKV: Semantic-Preserving KV Cache Compression. *arXiv:2502.00299*.

[10] Feng, S., et al. (2025). EvicPress: Joint KV-Cache Compression and Eviction. *arXiv:2512.14946*.

[11] Zhang, Y., et al. (2025). ThinKV: Thought-Adaptive KV Cache Compression for Efficient Reasoning. *arXiv:2510.01290*.

[12] Yang, K., et al. (2025). MiniKV: Pushing the Limits of 2-Bit KV Cache via Compression and System Co-Design. *arXiv:2411.18077*.